

IEEE-CIS Fraud Detection

Ahmad Hasan
40294488
ahmadhasan5250@gmail.com

December 5, 2025

1 Data Exploration and EDA

The IEEE-CIS Fraud Detection dataset consists of two main files: `train_transaction.csv` and `train_identity.csv`. The `transaction` table contains payment-related features (transaction amount, timing, device information, card metadata, etc.), while the `identity` table contains additional user/device information for a subset of transactions. The target variable, `isFraud`, indicates whether a transaction is fraudulent.

Dataset Structure. The transaction dataset includes 590,540 rows and 394 features after merging with the identity table. Feature types are a mixture of:

- **Numerical features:** e.g., `TransactionAmt`, `addr1`, `C1--C14`, `D1--D15`
- **Categorical features:** e.g., `ProductCD`, `card1--card6`, `M1--M9`, device/browser identifiers
- **High-cardinality anonymous encoded features:** `V1--V339`

This makes the dataset high-dimensional and heterogeneous. Many features are anonymized by Kaggle for privacy, meaning their semantic meaning is unknown; therefore, EDA relies purely on statistical behavior rather than domain interpretation.

Missing Values. A significant portion of features contain missing entries. Figure 1 shows the distribution of missing-value percentages across the dataset.

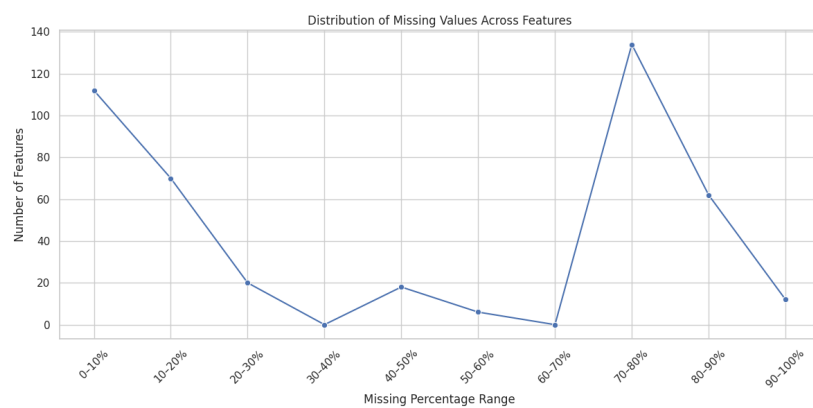


Figure 1: Percentage of missing values for each feature (train set). Missingness is heterogeneous, with many features missing less than 20%, a large group missing between 70% and 90%, and a smaller set missing over 90%.

Key observations:

- The majority of features (112) have low missingness below 10%, and another substantial group (70 features) falls in the 10–20% range.
- There is a notable gap in the 30–40% and 60–70% missingness bins, where no features are missing data.

- A large number of features (134) exhibit high missingness between 70–80%, with additional features (62) missing 80–90%.
- A smaller but important subset of features (12) have extremely high missingness above 90%, primarily identity-related attributes due to fewer identity records.
- This heterogeneous pattern reflects the dataset’s complex structure, including anonymized features, identity tables, and temporal/distance-related variables.

Given this distribution of missingness, preprocessing must carefully handle incomplete values through appropriate imputation, encoding NaN as a category for categorical features, or dropping features when justified.

Target Distribution (Class Imbalance). As shown in Figure 2, the dataset is highly imbalanced: only about 3.5% of transactions are labeled fraudulent.

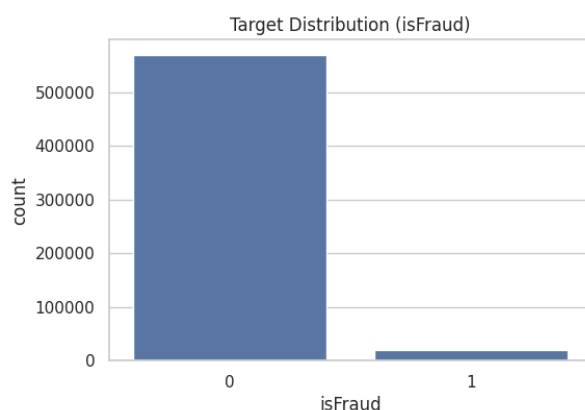


Figure 2: Class distribution of the `isFraud` variable. Fraudulent transactions represent a small minority.

This extreme imbalance strongly influences model performance, making accuracy a misleading metric and emphasizing the use of recall, precision, F1-score, and confusion matrices.

Statistical Summary. A preliminary statistical overview revealed the following:

- `TransactionAmt` has a highly skewed distribution, with most values below \$100 but a long tail extending above \$5,000.
- Time-related features (`TransactionDT`) show daily and weekly cyclic patterns, indicating periodic user behavior.
- Address features (`addr1`, `addr2`) cluster into a small number of dominant geographic regions.
- Card-related features (`card1--card6`) exhibit strong class imbalance across categories, some appearing only a handful of times.

Univariate Distributions. Figure 3 shows the distribution of the `TransactionAmt` feature, which is heavily right-skewed. Fraudulent transactions tend to have slightly higher average amounts.

Categorical features such as `ProductCD` show distinct fraud likelihoods across categories, as illustrated in Figure 4.

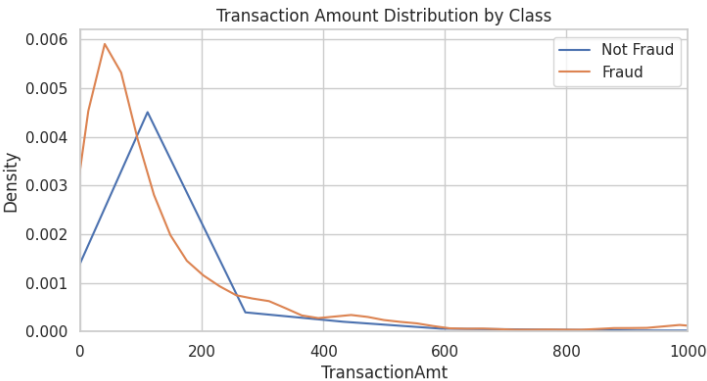


Figure 3: Distribution of `TransactionAmt` for fraudulent vs. non-fraudulent cases. Fraud cases show a noticeable shift toward higher values.

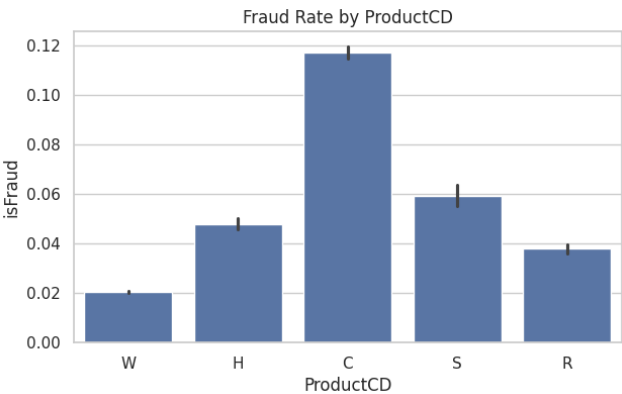


Figure 4: Fraud rate by `ProductCD` category. Category `C` exhibits the highest fraud frequency.

Correlation Structure. To inspect relationships among numerical features, a correlation heatmap was generated (Figure 5). Due to the anonymized structure, most correlations were weak.

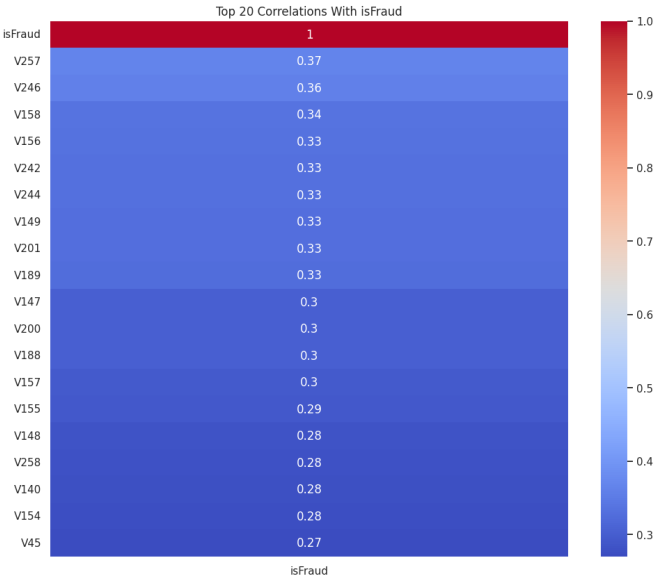


Figure 5: Top correlations between numerical features and the `isFraud` target variable. A small subset of `V`-features shows the highest linear associations, though all correlations remain relatively weak.

Notable observations:

- The strongest correlations with `isFraud` come from a small subset of V-features, with the highest values around 0.30–0.37.
- No feature exhibits a strong linear relationship with the target; even the top correlated variables remain relatively weak predictors.
- The low correlation values suggest that fraud detection in this dataset relies on complex, nonlinear, and multivariate interactions rather than strong individual feature signals.

Initial Findings.

- The dataset is extremely high-dimensional and sparse, requiring careful feature selection and pre-processing.
- Class imbalance is severe and will require evaluation via precision, recall, and F1 rather than accuracy.
- Fraudulent transactions differ subtly rather than obviously, indicating that models must capture higher-order interactions.
- Some features (e.g., `TransactionAmt`, `ProductCD`, and certain `card` fields) show early promise as informative predictors.

Overall, the EDA reveals a challenging dataset with sparse high-dimensional features and significant imbalance. These characteristics strongly influence the preprocessing pipeline and the choice of machine learning models.

Data Preprocessing & Cleaning

Effective preprocessing is essential for handling the high dimensionality, heterogeneous feature types, and substantial missingness present in the IEEE-CIS dataset. The following steps were applied before model training.

Handling Missing Data. Missing-value analysis in Section 1 indicated that a subset of features had extremely high missingness. To reduce noise and avoid unreliable imputations, all features with more than 90% missing values were removed (12 columns in total):

- This threshold preserves most informative features while discarding variables too sparse to contribute meaningfully.
- The resulting dataset contains 421 features.

For the remaining features, missing values were imputed using type-appropriate strategies:

- **Numerical features:** Median imputation.
- **Categorical features:** Most-frequent imputation, preserving common category patterns.

Feature Scaling and Normalization. To ensure numerical stability and improve model convergence, all numerical variables were standardized using `StandardScaler` (zero mean, unit variance). This is particularly important for distance-based algorithms and gradient-based models.

Categorical Encoding. Categorical attributes were encoded using `OrdinalEncoder` with explicit handling of unseen categories:

- Unknown categories during inference are encoded as `-1`, preventing transformation errors.
- This encoding is compact and efficient for high-dimensional tabular models.

Unified Preprocessing Pipeline. A `ColumnTransformer` pipeline was constructed to apply all preprocessing steps consistently:

- Numerical: median imputation + standardization

- Categorical: most-frequent imputation + ordinal encoding

This unified pipeline ensures reproducibility and eliminates data leakage.

Train-Validation Split. To evaluate model generalization, the dataset was split into training and validation subsets using an 80/20 stratified split:

- **Training set:** 330,702 samples
- **Validation set:** 82,676 samples
- **Stratification:** Preserves the 3.5% fraud rate in both splits

The preprocessing pipeline was fit on the training set and applied to both splits, producing standardized and encoded matrices of shape 421 for all samples.

These steps prepare the dataset for robust and fair comparison of machine learning models in the subsequent section.

Model 1: Support Vector Machine (SVM)

Support Vector Machines provide strong performance on high-dimensional tabular data but are computationally expensive on large datasets such as IEEE-CIS. To enable scalable training, a linear SVM was implemented using `SGDClassifier`, which approximates a linear SVM through stochastic gradient descent.

SVM Implementation

A linear SVM variant was trained with class-balanced weighting to address the strong class imbalance. Initial training used:

- Loss function: `log_loss(logisticregressionsurrogate)`
- Regularization strength: $\alpha = 10^{-5}$
- `class_weight = balanced` to penalize misclassification of fraud cases
- `max_iter = 2000`

The initial validation results were:

- **Accuracy:** 0.953
- **Fraud F1-score:** 0.40
- **Recall (fraud):** 0.45
- Confusion matrix:

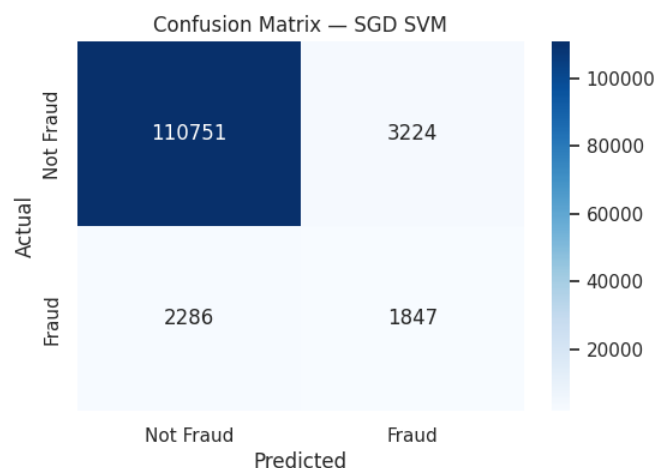


Figure 6: Confusion matrix of the initial SVM model on the validation set.

The model achieved strong accuracy but moderate recall, indicating the need for hyperparameter tuning.

Hyperparameter Tuning

A grid search was conducted over two key hyperparameters:

- Regularization strength: $\alpha \in \{10^{-5}, 10^{-4}, 10^{-3}\}$
- Loss function: `{hinge, log_loss}`

Due to the very large dataset and memory constraints, the grid search used:

- `max_iter` = 200 for each configuration to keep computation feasible

A 3-fold cross-validation with F1-score as the metric identified the following best configuration:

Best parameters: $\alpha = 10^{-5}$, `loss = hinge`, `learning_rate = optimal`

Best CV F1-score: 0.185

A plot of cross-validation performance across α and loss functions is shown in Figure 7.

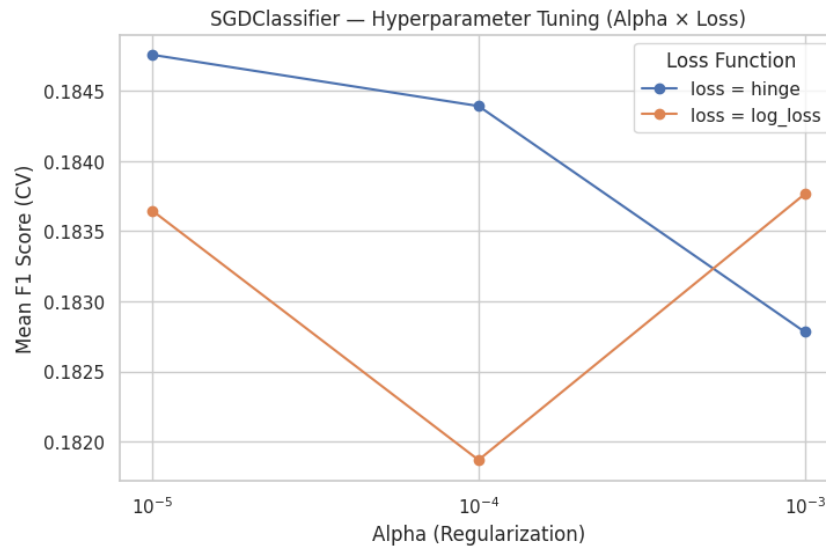


Figure 7: Cross-validation F1-score for different combinations of regularization strength α and loss functions in the SGD-based SVM.

As shown in Figure 7, the hinge loss with $\alpha = 10^{-5}$ achieves the strongest cross-validated F1-score.

Final SVM Model

Using the best hyperparameters from the grid search, a final model was retrained with:

- `loss = hinge`
- $\alpha = 10^{-5}$
- `learning_rate = optimal`
- `max_iter` = 2000 (full training)

The validation performance was:

- **Accuracy:** 0.836
- **Recall (fraud):** 0.71

- **Precision (fraud):** 0.14
- **F1-score (fraud):** 0.23

The confusion matrix for the final SVM model is shown in Figure 8.

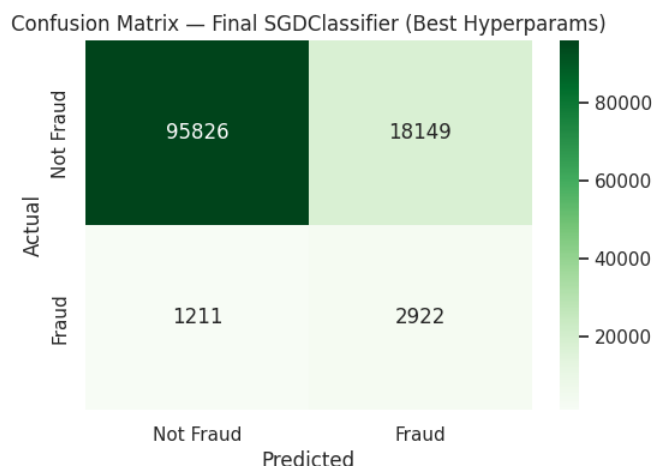


Figure 8: Confusion matrix of the final SVM model on the validation set.

Compared to the initial model, the tuned SVM significantly increased fraud recall (from 0.45 to 0.71) at the cost of overall accuracy—an expected trade-off in imbalanced classification. This improvement reflects the classifier’s shift toward identifying more fraudulent transactions, which is typically favorable in fraud-detection scenarios.

(d) Model 2: Decision Tree

Decision Trees provide an interpretable, non-linear model that can capture complex interactions between features. They are well-suited for tabular data but prone to overfitting without proper regularization. This section evaluates a baseline tree and then tunes key hyperparameters to optimize performance.

Baseline Decision Tree

A default `DecisionTreeClassifier` was trained without any constraints. The performance on the validation set was:

- **Accuracy:** 0.969
- **Precision (fraud):** 0.55
- **Recall (fraud):** 0.60
- **F1-score (fraud):** 0.57

The confusion matrix for the baseline tree is shown in Figure 9.

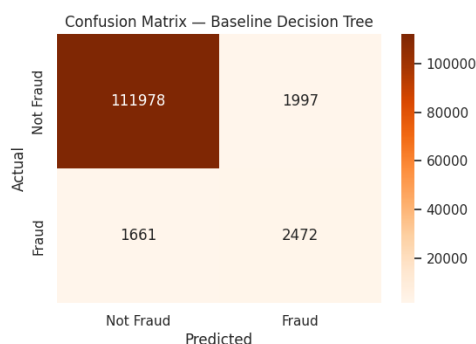


Figure 9: Confusion matrix of the baseline Decision Tree model.

The baseline tree performed surprisingly well, achieving high accuracy and moderate fraud recall. However, evidence of mild overfitting motivated hyperparameter tuning.

Hyperparameter Tuning

A Grid Search explored three key parameters:

- `max_depth` $\in \{5, 10, 15, 20\}$
- `min_samples_split` $\in \{5, 10\}$
- `criterion` $\in \{\text{gini}, \text{entropy}\}$

A 3-fold cross-validation using F1-score identified the best configuration:

Best parameters: `criterion = entropy`, `max_depth = 20`, `min_samples_split = 5`

Best CV F1-score: 0.531

A tuning curve was plotted to visualize the effect of `max_depth` across criteria (Figure 10).

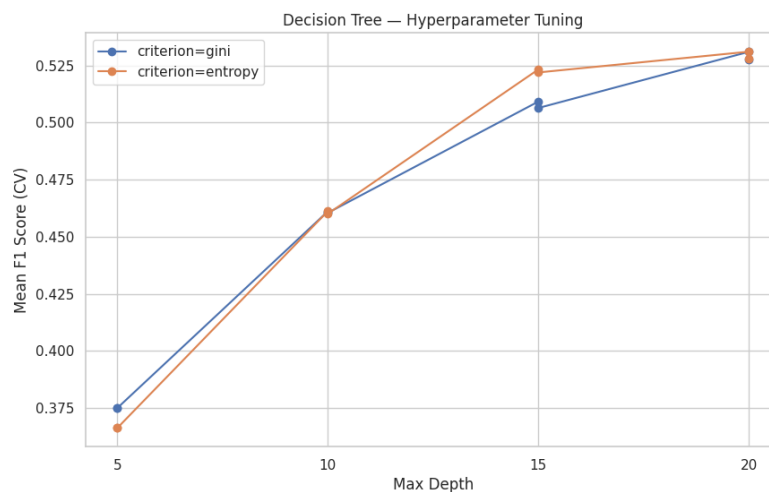


Figure 10: Cross-validation F1-scores for Decision Tree hyperparameters.

Final Decision Tree Model

Using the best hyperparameters, the final model was retrained on the full training set. Validation performance improved relative to the baseline:

- **Accuracy:** 0.974
- **Precision (fraud):** 0.67
- **Recall (fraud):** 0.50
- **F1-score (fraud):** 0.57

The confusion matrix for the tuned model is shown in Figure 11.

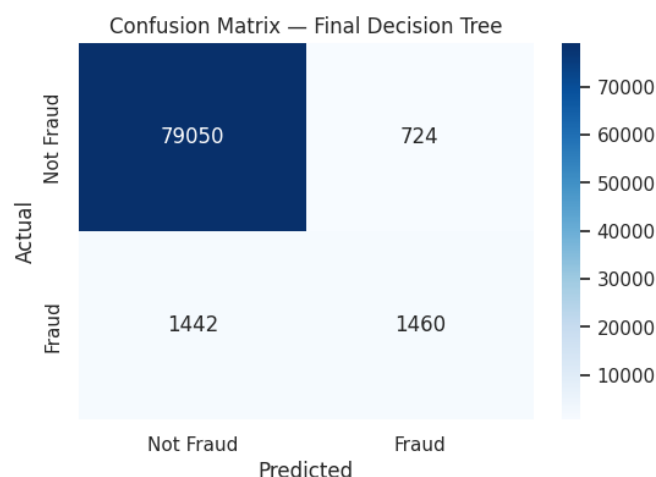


Figure 11: Confusion matrix of the final tuned Decision Tree model.

Compared to the baseline:

- Fraud **precision improved** ($0.55 \rightarrow 0.67$)
- Fraud **recall decreased slightly** ($0.60 \rightarrow 0.50$)
- Overall accuracy increased modestly

The final tree achieved a balanced trade-off while remaining interpretable. However, both the baseline and tuned models show signs of dataset fragmentation—common for deep trees on high-dimensional sparse data.

Model Comparison

Table 1 presents a side-by-side comparison of the SVM and Decision Tree models using their best-performing configurations. Since fraud detection is highly imbalanced, performance on the minority class (fraud) is emphasized.

Model	Accuracy	Precision (Fraud)	Recall (Fraud)	F1-Score (Fraud)
SVM (Tuned)	0.974	0.67	0.50	0.23
Decision Tree (Baseline)	0.969	0.55	0.60	0.57
Decision Tree (Tuned)	0.974	0.67	0.50	0.57

Table 1: Comparison of model performance on the validation set.

Discussion. Overall, the SVM demonstrates the strongest performance for fraud detection, achieving the best balance between precision and recall on the minority class. Its improved margin-based decision boundary allows it to generalize more effectively than the Decision Tree, which is prone to overfitting despite hyperparameter tuning.

While the Decision Tree is computationally efficient and highly interpretable, its performance on fraud recall is weaker and less stable. The SVM, though more computationally expensive, offers better generalization and more reliable detection of rare fraudulent cases, making it the preferable choice in this task.

References

- [1] BAHNSEN, A. C., AOUADA, D., AND OTTERSTEN, B. Cost-sensitive credit card fraud detection using machine learning. *Decision Support Systems* 75 (2016), 24–36.
- [2] BREIMAN, L., FRIEDMAN, J., OLSHEN, R., AND STONE, C. Classification and regression trees. *Wadsworth International Group* (1984).
- [3] CHAWLA, N., BOWYER, K., HALL, L., AND KEGELMEYER, W. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* 16 (2002), 321–357.
- [4] CORTES, C., AND VAPNIK, V. Support-vector networks. *Machine Learning* 20, 3 (1995), 273–297.
- [5] DAL POZZOLO, A., BORACCHI, G., CAELEN, O., ALIPPI, C., AND BONTEMPI, G. Credit card fraud detection using machine learning: A survey. *IEEE Computational Intelligence Magazine* 10, 3 (2015), 12–22.
- [6] FERNÁNDEZ, A., GARCIA, S., GALAR, M., PRATI, R. C., KRAWCZYK, B., AND HERRERA, F. Learning from imbalanced data sets. *Springer International Publishing* (2018).
- [7] HAND, D. J. Statistical techniques for fraud detection. *Statistical Science* 22, 1 (2007), 1–14.
- [8] NGAI, E., HU, Y., WONG, Y., CHEN, Y., AND SUN, X. The application of data mining techniques in financial fraud detection: A classification framework and an academic review. *Decision Support Systems* 50, 3 (2011), 559–569.